23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : **A8**

Team Members :

Roll No	Student Name
AM.SC.U4CSE25014	ADITHYAN B
AM.SC.U4CSE25040	MADHAV A
AM.SC.U4CSE25049	ROBIN ANTONY
AM.SC.U4CSE25024	ALI ADNAN THAHA

Project Title :

Smart Digital Library Management System (BookVerse)

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	3
2	Introduction	4-7
3	Requirements (Functional & Non-functional)	8-10
4	UI Design	11-16
5	Database Design	17-26
6	Database Selection & Justification	27-28
7	Implementation	29-35
8	Testing & Validation	36-38
9	Results & Screenshots	39-41
10	Challenges faced & solutions	42-43
11	Conclusion & future enhancements	44-45
	References	46

1. ABSTRACT

The Smart Digital Library Management System, named BookVerse, is a database-driven application developed to simplify and organize the major activities of a college library. Traditional library management methods can lead to difficulties in maintaining student records, tracking book availability, managing borrowing transactions, and monitoring reservations and overdue books. BookVerse addresses these challenges by providing a centralized application for managing library resources and student information. The system is designed around two primary user roles: Library Administrator and Student. The administrator can manage library records and student information, while students can access the available book catalog and view relevant borrowing information. PostgreSQL is used as the relational database management system, Java with Spring Boot is used for backend development, and HTML, CSS, and JavaScript are used to develop the user interface. The database design includes entities such as Student, Book, Author, Publisher, Category, Book Copy, Borrow Transaction, Reservation, and Fine. Primary keys, foreign keys, and other constraints help maintain data integrity and establish relationships between records. The project demonstrates important DBMS concepts, including relational database design, functional dependencies, normalization, SQL operations, and database connectivity. BookVerse aims to improve the organization, accuracy, and accessibility of library information while reducing the manual effort involved in library administration.

2. INTRODUCTION

2.1 Background

A library is an important part of an educational institution because it provides students and faculty members with access to books and other learning resources. Managing a library involves several interconnected activities, including maintaining book details, registering students, recording book issues and returns, tracking physical copies, handling reservations, and maintaining borrowing records.

When these activities are managed manually or through disconnected systems, it becomes difficult to maintain accurate and up-to-date information. Library staff may need to search through multiple records to determine whether a book is available, identify overdue books, or retrieve the borrowing history of a particular student. Repeatedly entering the same information can also introduce inconsistencies and data redundancy.

A database management system can address these challenges by storing related information in a structured format and providing mechanisms to retrieve and update records efficiently. A well-designed relational database also helps maintain relationships between entities and prevents invalid references between records.

2.2 Problem Statement

The main problem addressed by this project is the difficulty of managing library information through manual or insufficiently integrated methods. Book details, student information, borrowing records, and reservations are related to one another, making it necessary to maintain consistent records across the system.

The major problems include:

- Difficulty in maintaining and updating book and student records.
- Time-consuming searches for books and their availability.
- Challenges in tracking individual physical copies of books.
- Difficulty in maintaining borrowing and return histories.

- Possibility of duplicate or inconsistent data.
- Challenges in identifying overdue transactions and managing fines.
- Limited access to organized information for administrative decisions.

These issues highlight the need for a centralized library management application supported by a properly designed relational database.

2.3 Proposed Solution

BookVerse is proposed as a Smart Digital Library Management System that combines a web-based user interface, a backend application, and a relational database.

The user interface provides separate experiences for administrators and students. The backend processes application requests and communicates with the database to retrieve or modify information. PostgreSQL stores the structured data and maintains relationships between records using primary keys and foreign keys.

The application is organized into modules for book catalog management, student management, borrowing information, reservations, and related library operations. The system is designed to support data insertion, retrieval, modification, deletion, and searching wherever applicable.

2.4 Objectives

The major objectives of BookVerse are:

1. To develop a centralized system for managing library information.
2. To design a relational database containing the major entities involved in library operations.
3. To establish relationships between books, authors, publishers, categories, students, and transactions.
4. To apply primary keys, foreign keys, and constraints to maintain data integrity.
5. To reduce unnecessary data redundancy through normalization.
6. To develop an interface that separates administrator and student activities.

7. To implement database operations through a backend application.
8. To demonstrate practical DBMS concepts through a working application.
9. To provide a foundation for future improvements in library reporting, automation, and resource management.

2.5 Scope of the Project

The project focuses on the design and implementation of a database-driven college library management application. Its database design covers student records, books, authors, publishers, categories, physical book copies, borrowing transactions, reservations, and fines.

The system also includes a user interface for accessing library functions and a backend layer for processing application requests. The scope includes database connectivity, structured data storage, record management, and the implementation of relevant library workflows.

Advanced functions such as comprehensive analytics, automated notifications, and integration with external library services can be extended in future versions.

2.6 Existing System and Proposed System

The existing manual or disconnected approach requires library staff to maintain and retrieve information through separate records or tools. This can make searching, updating, and tracking related transactions difficult.

The proposed system organizes the information in a relational database and provides a centralized application interface.

Existing Approach	Proposed BookVerse System
Records may be maintained separately	Related records are stored in a structured database
Searching may require manual effort	Application-based catalog search and retrieval

Updating records can be repetitive	Centralized record management
Relationships between records may be difficult to track	Primary keys and foreign keys establish relationships
Borrowing information may be distributed across records	Borrowing transactions are represented as database records
Reports may require manual compilation	The design provides a foundation for organized reporting

3. REQUIREMENTS (FUNCTIONAL AND NON-FUNCTIONAL)

3.1 Functional Requirements

Functional requirements describe the operations that the system is designed to perform.

1. User Authentication

The system provides a login interface intended to distinguish between Library Administrators and Students. Access to application functions is organized according to the user's role.

2. Book Catalog Management

The administrator can manage book information, including the title, ISBN, publication year, category, and publisher. The system is designed to support creating, viewing, updating, and deleting book records.

3. Student Management

The system maintains student information such as Student ID, name, email, phone number, and department. Administrative functions support viewing and maintaining student records.

4. Book Search and Filtering

Users can search the catalog and filter books according to relevant information, such as title, author, category, and availability, as supported by the application.

5. Book Copy Management

The database represents physical copies separately from book titles. Each copy has an accession number and a status, allowing individual copies to be tracked.

6. Borrowing and Return Management

The database design includes borrowing transactions containing the student, physical copy, issue date, due date, return date, and transaction status.

7. Reservation Management

The design includes reservations associated with students and books. Reservation status is maintained to support the reservation workflow.

8. Fine Management

The database design includes fine records associated with borrowing transactions. The amount and payment status are stored for tracking outstanding or paid fines.

9. Borrowing History

The system is designed to maintain transaction records that can be used to retrieve the borrowing history of a student.

10. Reporting and Dashboard

The UI design includes an administrative dashboard and reporting interface. These are intended to provide an overview of library activity, including circulation information and collection status.

3.2 Non-Functional Requirements

1. Usability

The interface should be understandable and allow users to perform common library operations with minimal unnecessary navigation.

2. Responsiveness

The interface should adapt to different screen widths, including desktop, tablet, and mobile layouts.

3. Performance

Common catalog searches and filtering operations should provide timely results for the available dataset.

4. Data Integrity

The database should use suitable primary keys, foreign keys, uniqueness rules, and required-field constraints to maintain valid records.

5. Security

Authentication and role-based access should restrict administrative operations to authorized users. Database credentials should be configured securely and should not be published in the public repository.

6. Maintainability

The application separates HTML structure, CSS styling, JavaScript interaction logic, backend processing, and database storage into distinct layers.

7. Reliability

The application should handle invalid inputs and database errors appropriately without corrupting existing records.

8. Scalability

The relational database and modular application structure provide a foundation for extending the catalog and adding more library operations.

4. UI DESIGN

4.1 Overview

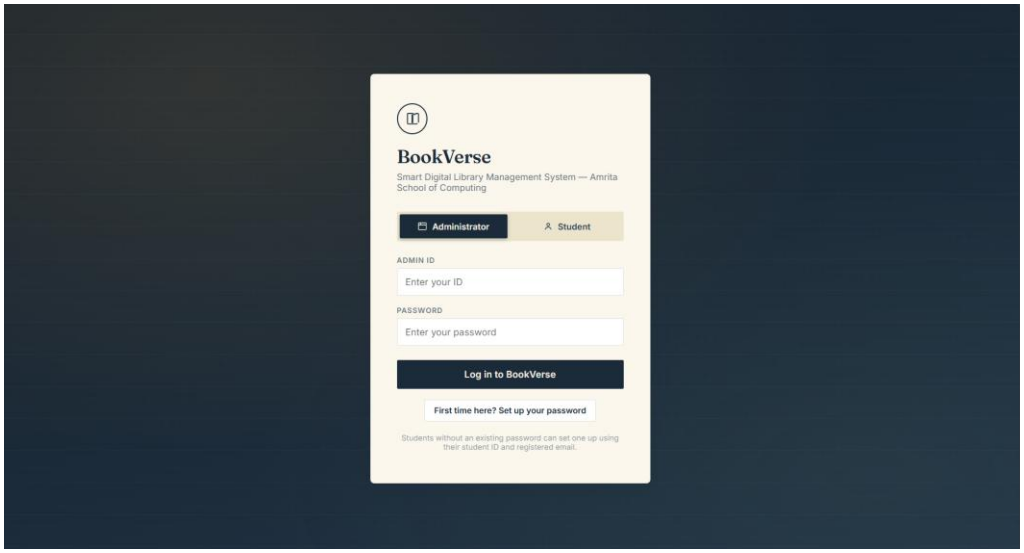
The user interface of BookVerse is designed to provide a clear and organized way to interact with library information. The design separates administrative operations from student-facing functions and uses a consistent visual style across the application.

The interface is implemented using HTML, CSS, and JavaScript. The HTML defines the structure of the pages, CSS controls the visual appearance, and JavaScript handles interactions, navigation, form operations, and dynamic rendering. Chart.js is included in the documented UI design for dashboard and reporting visualizations.

The application uses a card-based catalog presentation to make book information easier to browse.

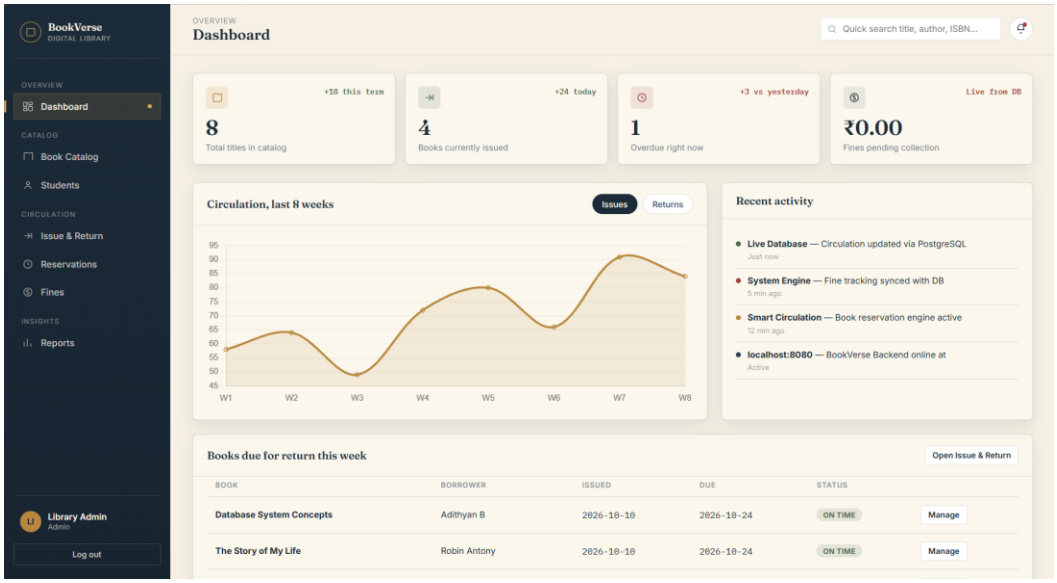
4.2 Login Interface

The login screen provides the entry point to the application. It is designed around the two principal user roles: Library Administrator and Student. After authentication, the user should access the functions appropriate to the selected role. Administrative functions include management activities, whereas student functions focus on catalog access and personal library information.



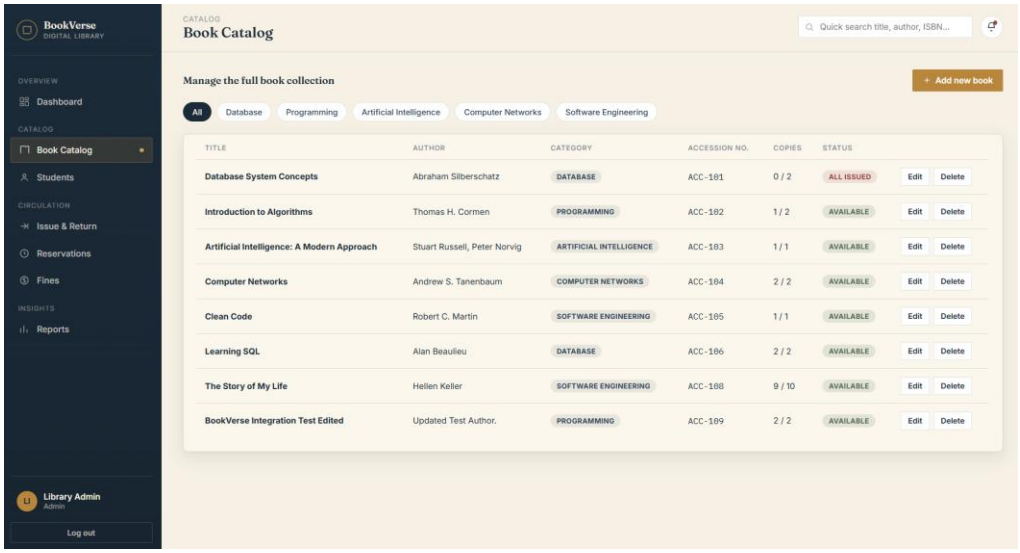
4.3 Administrator Dashboard

The administrator dashboard is designed to provide a summary of library activity. The UI design document includes summary statistics, a circulation chart, and a due-soon table. The dashboard acts as a starting point for administrative tasks and provides navigation to the relevant management modules.



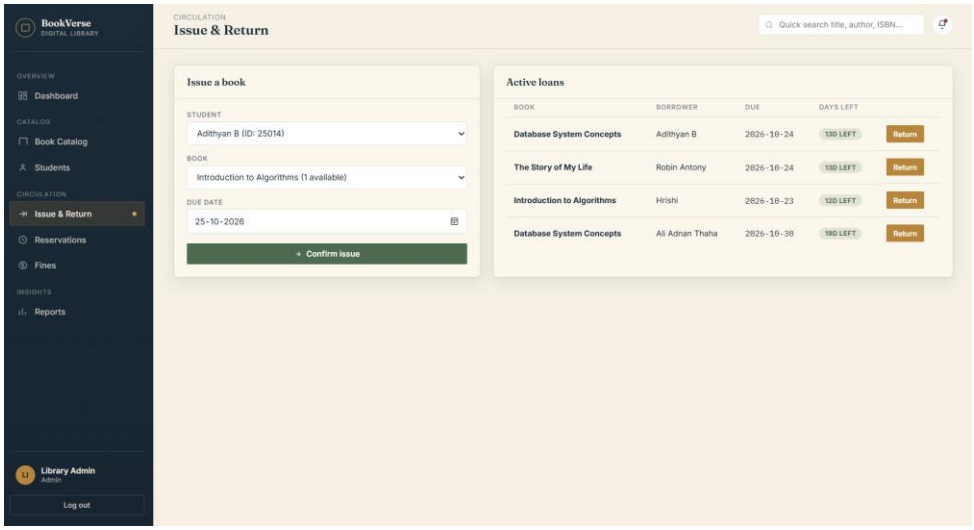
4.4 Book Catalog Management

The book management interface is designed to support the maintenance of library catalog information. Administrators can work with book details such as title, ISBN, publication year, category, and publisher. The interface supports the catalog management workflow through forms and record displays. The backend and database are responsible for processing and storing the corresponding changes.



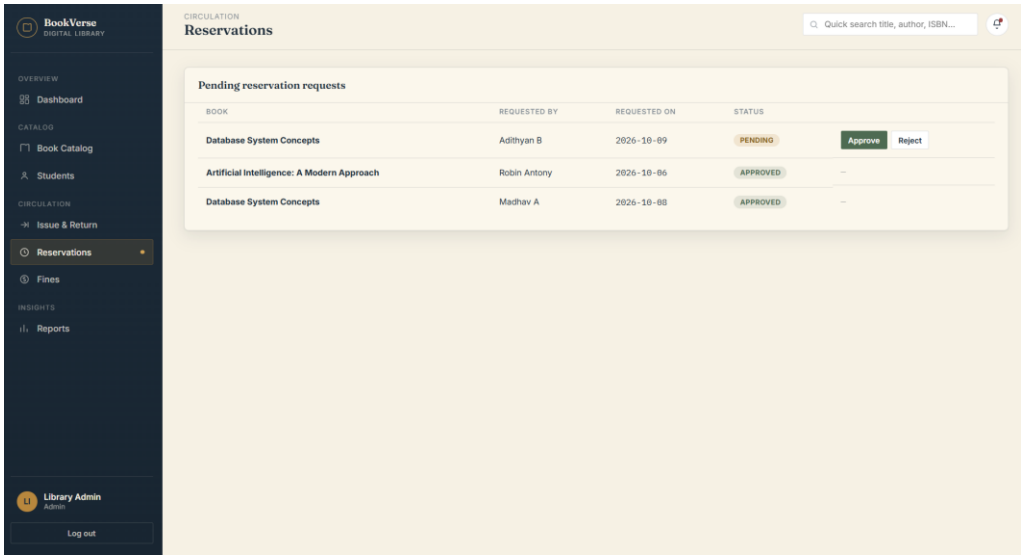
4.5 Issue and Return Interface

The issue and return interface is designed to represent borrowing activity and active loans. The database model associates each borrowing transaction with a student and a physical book copy.The transaction record contains issue, due, and return dates along with a status. These attributes provide the information needed to track the lifecycle of a borrowing transaction.



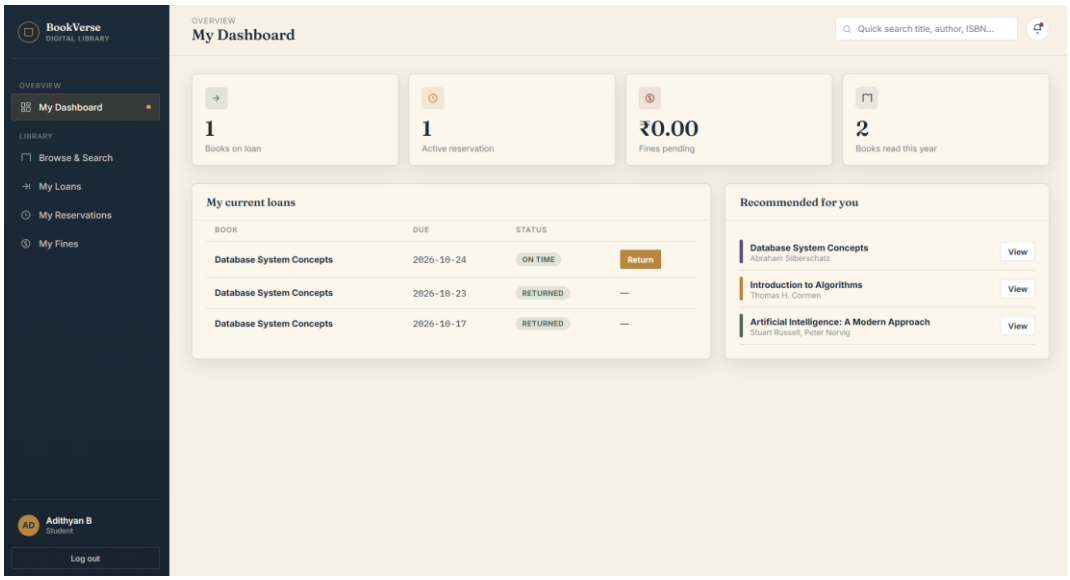
4.6 Reservation Interface

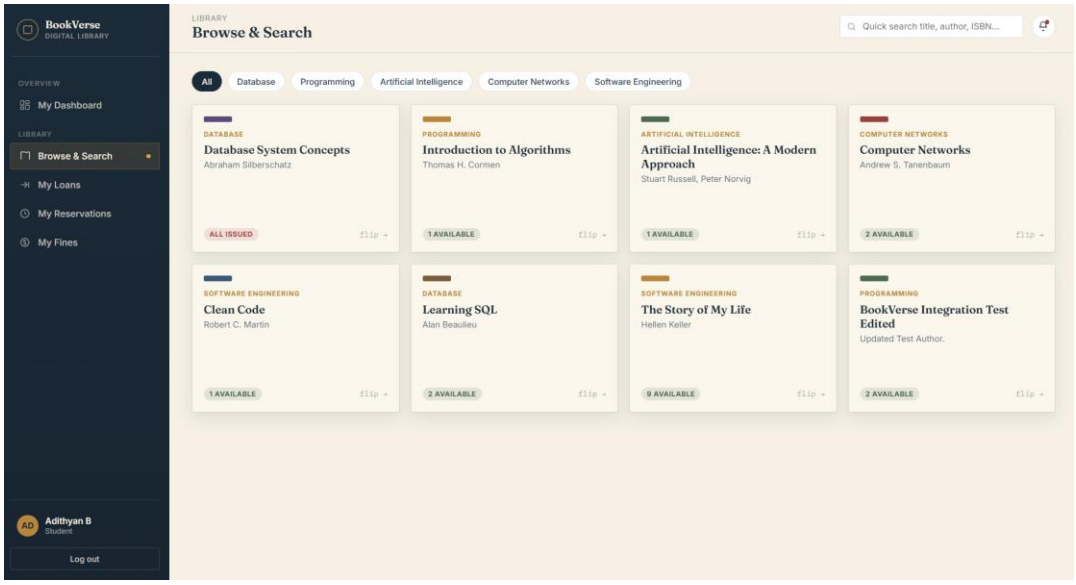
The reservation interface is designed to present reservation records and support administrative handling of reservations. The RESERVATION relation associates a reservation with a student and a book. The reservation date and status help represent the progress of the reservation.



4.7 Student Dashboard and Catalog

The student-facing interface focuses on book discovery and personal library information. The UI design document presents the catalog using interactive, flip-style index cards. The catalog presentation is intended to make book details easy to browse. Relevant book information and available actions can be displayed through the interface, depending on the state of the record and the implemented application workflow.

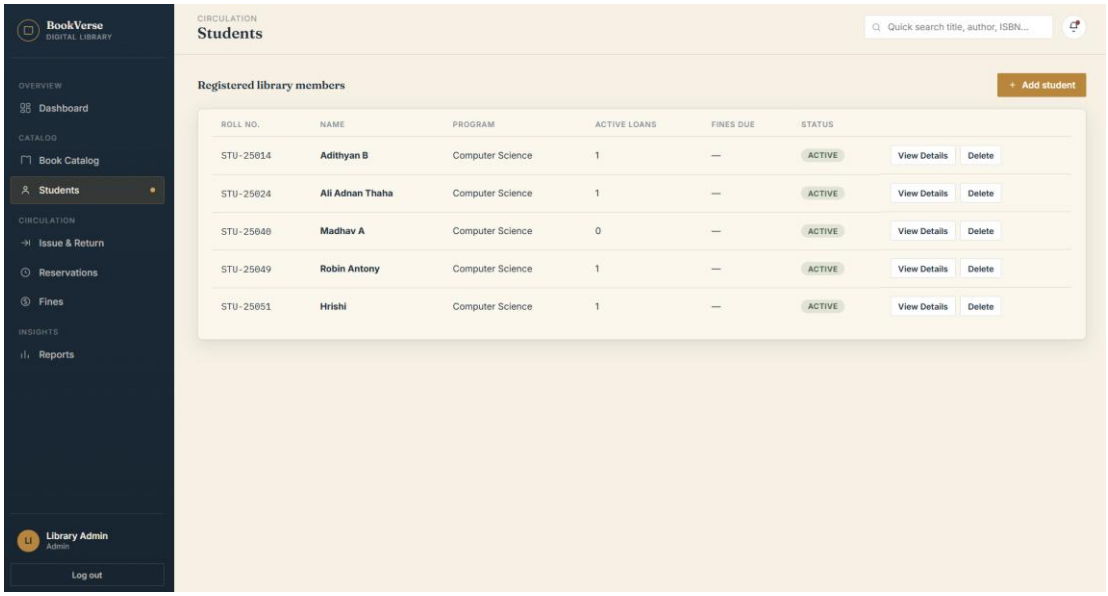




4.8 Student Management Interface

The student management interface allows administrative users to view student records and access the available student-management actions. The student information includes the student's identifier, name, email, phone number, and department.

The implemented student-management workflow also includes viewing student details and a deletion operation with backend checks for related borrowing history or reservations.



4.9 Reports Interface

The documented UI design includes a reporting interface for presenting library-related information. The proposed reports cover areas such as circulation trends, popular categories, and collection status.



5. DATABASE DESIGN

5.1 Overview

Database design is a central component of BookVerse because the application contains several entities whose information is interrelated. A relational database is used to organize this information into tables and establish relationships between them.

The database design document identifies nine principal relations: STUDENT, CATEGORY, PUBLISHER, BOOK, AUTHOR, BOOK_AUTHOR, BOOK_COPY, BORROW_TRANSACTION, RESERVATION, and FINE.

The design uses primary keys to uniquely identify records and foreign keys to connect related records. It also applies normalization principles to reduce redundancy and avoid common insertion, update, and deletion anomalies.

5.2 Entities and Attributes

1. STUDENT

Stores information about students registered in the library.

Attributes:

- Student_ID — Primary Key
- Name
- Email
- Phone
- Department

Student_ID uniquely identifies each student. Email is intended to be unique.

2. CATEGORY

Stores the categories used to classify books.

Attributes:

- Category_ID — Primary Key
- Category_Name

A category can be associated with multiple books.

3. PUBLISHER

Stores information about publishers.

Attributes:

- Publisher_ID — Primary Key
- Publisher_Name

A publisher can publish multiple books.

4. BOOK

Stores general information about book titles.

Attributes:

- Book_ID — Primary Key
- Title
- ISBN
- Publication_Year
- Category_ID — Foreign Key
- Publisher_ID — Foreign Key

Book_ID identifies a book title. ISBN is also intended to be unique. Category_ID references CATEGORY, and Publisher_ID references PUBLISHER.

5. AUTHOR

Stores author information.

Attributes:

- Author_ID — Primary Key
- Author_Name

An author can be associated with multiple books.

6. BOOK_AUTHOR

Represents the many-to-many relationship between books and authors.

Attributes:

- Book_ID — Composite Primary Key component and Foreign Key
- Author_ID — Composite Primary Key component and Foreign Key

The combination of Book_ID and Author_ID forms the composite primary key. This relation allows one book to have multiple authors and one author to be associated with multiple books.

7. BOOK_COPY

Stores information about individual physical copies of a book.

Attributes:

- Accession_Number — Primary Key
- Book_ID — Foreign Key
- Status

A book title can have multiple physical copies. Accession_Number uniquely identifies each copy.

8. BORROW_TRANSACTION

Stores information about borrowing and returning physical copies.

Attributes:

- Borrow_ID — Primary Key
- Student_ID — Foreign Key
- Accession_Number — Foreign Key
- Issue_Date
- Due_Date
- Return_Date
- Status

Each borrowing transaction identifies the student and the physical book copy involved in the transaction.

9. RESERVATION

Stores reservation information.

Attributes:

- Reservation_ID — Primary Key
- Student_ID — Foreign Key
- Book_ID — Foreign Key
- Reservation_Date
- Status

A reservation is associated with a student and a book.

10. FINE

Stores fine information associated with borrowing transactions.

Attributes:

- Fine_ID — Primary Key
- Borrow_ID — Foreign Key
- Amount
- Payment_Status

The design specifies at most one fine per borrowing transaction, making Borrow_ID a candidate key in FINE as well.

5.3 Relational Schema

The relational schema derived from the database design is as follows:

1. STUDENT(**Student_ID**, Name, Email, Phone, Department)
2. CATEGORY(**Category_ID**, Category_Name)
3. PUBLISHER(**Publisher_ID**, Publisher_Name)

4. BOOK(**Book_ID**, Title, ISBN, Publication_Year, Category_ID, Publisher_ID)
5. AUTHOR(**Author_ID**, Author_Name)
6. BOOK_AUTHOR(**Book_ID**, **Author_ID**)
7. BOOK_COPY(**Accession_Number**, Book_ID, Status)
8. BORROW_TRANSACTION(**Borrow_ID**, Student_ID, Accession_Number, Issue_Date, Due_Date, Return_Date, Status)
9. RESERVATION(**Reservation_ID**, Student_ID, Book_ID, Reservation_Date, Status)
10. FINE(**Fine_ID**, Borrow_ID, Amount, Payment_Status)

Note: The schema above represents the submitted database design. The actual deployed schema should be checked against the PostgreSQL database and the SQL scripts included in the final repository before submission.

5.4 Primary Keys and Foreign Keys

A primary key uniquely identifies a row in a relation. For example, Student_ID identifies a student, Book_ID identifies a book title, and Borrow_ID identifies a borrowing transaction.

Foreign keys establish relationships between related relations.

The principal foreign-key relationships are:

- BOOK.Category_ID references CATEGORY.Category_ID.
- BOOK.Publisher_ID references PUBLISHER.Publisher_ID.
- BOOK_AUTHOR.Book_ID references BOOK.Book_ID.
- BOOK_AUTHOR.Author_ID references AUTHOR.Author_ID.
- BOOK_COPY.Book_ID references BOOK.Book_ID.
- BORROW_TRANSACTION.Student_ID references STUDENT.Student_ID.
- BORROW_TRANSACTION.Accession_Number references BOOK_COPY.Accession_Number.

- RESERVATION.Student_ID references STUDENT.Student_ID.
- RESERVATION.Book_ID references BOOK.Book_ID.
- FINE.Borrow_ID references BORROW_TRANSACTION.Borrow_ID.

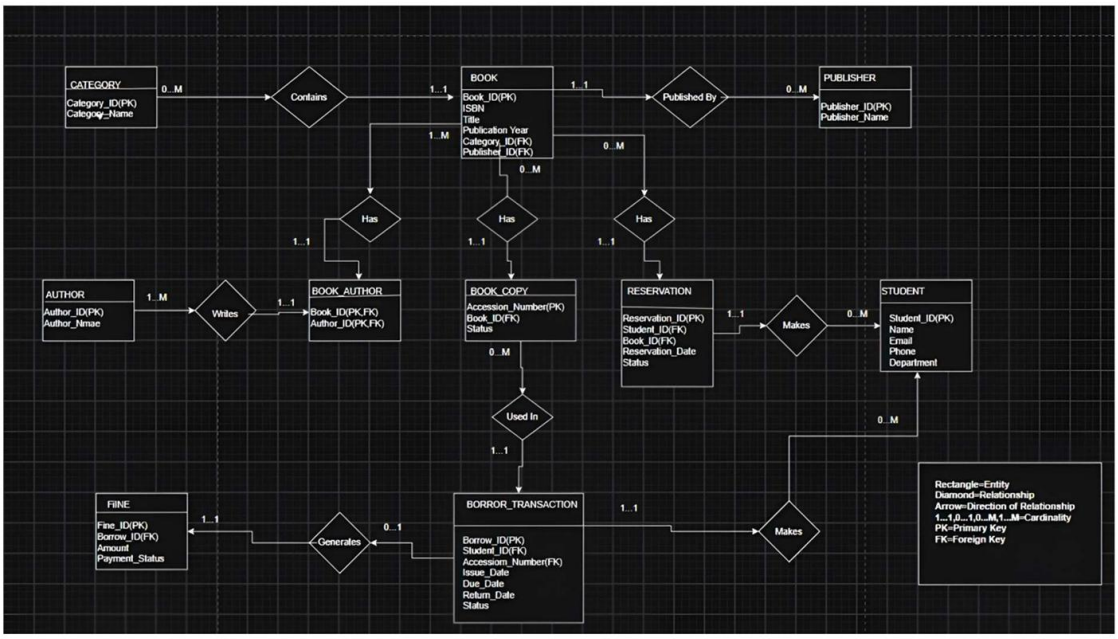
These relationships help maintain referential integrity by ensuring that referenced records exist.

5.5 ER Model and Relationships

The entity-relationship model represents the major entities and the relationships between them.

The important relationships include:

1. CATEGORY to BOOK: One category can contain multiple books, while each book belongs to a category.
2. PUBLISHER to BOOK: One publisher can publish multiple books.
3. BOOK to AUTHOR: Many-to-many relationship implemented through BOOK_AUTHOR.
4. BOOK to BOOK_COPY: One book title can have multiple physical copies.
5. STUDENT to BORROW_TRANSACTION: One student can have multiple borrowing transactions.
6. BOOK_COPY to BORROW_TRANSACTION: A physical copy can appear in borrowing records over time.
7. STUDENT to RESERVATION: One student can make multiple reservations.
8. BOOK to RESERVATION: A book can be associated with multiple reservation records.
9. BORROW_TRANSACTION to FINE: A borrowing transaction can have zero or one associated fine under the proposed design.



5.6 Conversion of ER Model into Relational Tables

The ER model is converted into relational tables using standard relational mapping principles.

Strong Entities

Each strong entity is converted into a separate relation. STUDENT, BOOK, AUTHOR, CATEGORY, and PUBLISHER are represented as independent tables with their own primary keys.

One-to-Many Relationships

For one-to-many relationships, the primary key of the one-side entity is included as a foreign key in the many-side relation. For example, Category_ID and Publisher_ID are stored in BOOK.

Many-to-Many Relationships

The relationship between BOOK and AUTHOR is represented by the BOOK_AUTHOR junction table. This table contains the keys of both entities and uses the combination of Book_ID and Author_ID as its primary key.

Physical Book Copies

The BOOK_COPY relation separates an individual physical copy from the general book title. This allows a single title to have several copies with separate accession numbers and statuses.

Transactional Relationships

BORROW_TRANSACTION references STUDENT and BOOK_COPY. RESERVATION references STUDENT and BOOK, while FINE references BORROW_TRANSACTION.

This mapping converts the conceptual model into a set of related tables that can be implemented using SQL.

5.7 Functional Dependencies

A functional dependency X → Y means that the attributes in X determine the attributes in Y.

The principal functional dependencies identified in the design are:

Relation	Functional Dependency
STUDENT	Student_ID → Name, Email, Phone, Department
CATEGORY	Category_ID → Category_Name
PUBLISHER	Publisher_ID → Publisher_Name
BOOK	Book_ID → Title, ISBN, Publication_Year, Category_ID, Publisher_ID
AUTHOR	Author_ID → Author_Name
BOOK_COPY	Accession_Number → Book_ID, Status
BORROW_TRANSACTION	Borrow_ID → Student_ID, Accession_Number, Issue_Date, Due_Date, Return_Date, Status
RESERVATION	Reservation_ID → Student_ID, Book_ID, Reservation_Date, Status

FINE	Fine_ID → Borrow_ID, Amount, Payment_Status
BOOK_AUTHOR	(Book_ID, Author_ID) is the composite key

The design also treats ISBN as a candidate key for BOOK because it is unique. In FINE, Borrow_ID is a candidate key when the rule of at most one fine per borrowing transaction is enforced.

These dependencies help identify candidate keys and verify that attributes are associated with the correct entities.

5.8 Normalization

Normalization is the process of organizing relational tables to reduce unnecessary redundancy and avoid data anomalies.

First Normal Form (1NF)

A relation satisfies 1NF when its attributes contain atomic values rather than repeating groups or multiple values stored in a single field.

The many-to-many relationship between books and authors is represented through BOOK_AUTHOR instead of storing several authors in one BOOK attribute. This keeps the relationship in a structured relational form.

Second Normal Form (2NF)

A relation satisfies 2NF when it is in 1NF and has no partial dependency of a non-key attribute on part of a candidate key.

BOOK_AUTHOR has a composite key consisting of Book_ID and Author_ID but contains no additional non-key attributes. The other principal relations use single-attribute primary keys in the submitted design. Therefore, the design avoids partial dependencies.

Third Normal Form (3NF)

A relation satisfies 3NF when it is in 2NF and has no inappropriate transitive dependencies of non-key attributes on a candidate key.

For example, Category_Name is stored in CATEGORY rather than being repeated in every BOOK record. Similarly, Publisher_Name is stored in

PUBLISHER. BOOK stores the corresponding foreign keys instead of duplicating the descriptive information.

This separation reduces repeated data and simplifies updates.

Boyce-Codd Normal Form (BCNF)

A relation satisfies BCNF when every determinant of a non-trivial functional dependency is a superkey.

According to the submitted functional dependency analysis, the determinants identified for the principal relations are candidate keys or superkeys. BOOK has Book_ID and ISBN as candidate keys under the uniqueness assumptions, while FINE has Fine_ID and Borrow_ID under the one-fine-per-borrow rule.

The submitted design therefore aims to satisfy BCNF. The implementation must enforce the relevant uniqueness constraints for these assumptions to hold.

5.9 Integrity Constraints

The design uses several types of integrity constraints:

- Entity integrity: Primary keys uniquely identify records and cannot be null.
- Referential integrity: Foreign keys link records to existing parent records.
- Domain integrity: Data types restrict values to appropriate formats.
- Uniqueness: Attributes such as student email, book ISBN, category name, and publisher name are intended to be unique where specified.
- Required attributes: Important fields such as student name, book title, ISBN, and relationship keys are intended to be mandatory.
- Business rules: A borrowing transaction must reference a valid student and physical copy. A fine must reference a valid borrowing transaction.

These constraints help prevent invalid or inconsistent database states.

6. DATABASE SELECTION AND JUSTIFICATION

6.1 Selected DBMS: PostgreSQL

PostgreSQL is the relational database management system selected for BookVerse.

The application requires a database capable of storing structured information, maintaining relationships among entities, enforcing constraints, and supporting SQL operations. PostgreSQL provides these capabilities and is suitable for implementing the relational schema of the library management system.

6.2 Reasons for Selecting PostgreSQL

1. Relational Data Model

The project contains related entities such as Student, Book, Author, Book Copy, Borrow Transaction, Reservation, and Fine. PostgreSQL supports relational tables and relationships between them.

2. Primary Keys and Foreign Keys

PostgreSQL supports primary-key, foreign-key, unique, and other constraints that help enforce data integrity.

3. SQL Support

The project uses SQL concepts such as table creation, data insertion, retrieval, updating, deletion, filtering, and joins. PostgreSQL supports these operations.

4. Transaction Support

Library operations may involve related database changes. Transaction support helps maintain consistency when multiple operations must be treated as one logical unit.

5. Open-Source Availability

PostgreSQL is open-source software, making it suitable for an academic project without requiring a commercial database licence.

6. Integration with Java

The application connects PostgreSQL to a Java Spring Boot backend. This architecture separates database storage from application processing and user-interface interaction.

7. Extensibility

PostgreSQL provides a foundation for future additions such as more complex reports, improved search functionality, and additional administrative modules.

6.3 Database Connectivity

The backend uses application configuration to identify the database connection details, including the database URL, username, and password.

The Spring Boot application establishes a connection to PostgreSQL using the configured database access mechanism. When an application operation requires database information, the backend processes the request and performs the corresponding database operation.

Database credentials must be configured for each development environment and should not be exposed in public source code.

7. IMPLEMENTATION

7.1 Technology Stack

The project uses the following technologies:

Component	Technology	Purpose
Frontend	HTML	Defines the page structure
Styling	CSS	Controls layout, typography, and appearance
Frontend logic	JavaScript	Handles interface interactions and dynamic rendering
Backend	Java with Spring Boot	Processes application requests and business logic
Database	PostgreSQL	Stores structured library information
Data access	Backend database connectivity layer	Connects application operations to persistent data
Version control	Git and GitHub	Maintains source code and project files

The documented UI design also uses Chart.js for dashboard and reporting visualizations.

7.2 System Architecture

BookVerse follows a layered application architecture.

Presentation Layer

The presentation layer consists of HTML, CSS, and JavaScript. It displays the login screen, catalog, management interfaces, and other application views. It also collects user input and presents the results returned by the backend.

Application Layer

The backend is developed using Java and Spring Boot. It receives application requests, processes the required operations, validates relevant input, and communicates with the database.

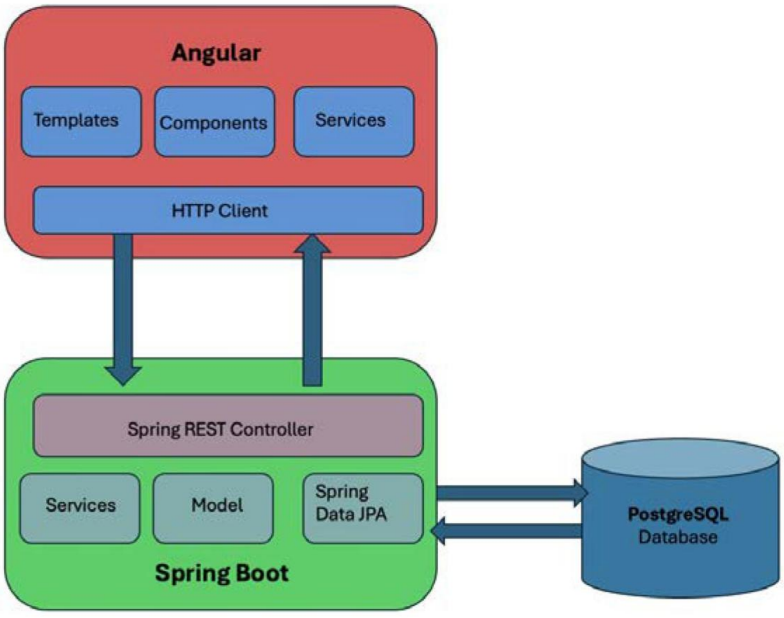
Database Layer

PostgreSQL stores the application's structured records. The database contains related entities and uses keys and constraints to maintain the relationships between them.

Request Flow

The general request flow is:

1. A user performs an action through the frontend.
2. The frontend sends the relevant request to the backend.
3. The backend validates and processes the request.
4. The backend communicates with PostgreSQL.
5. PostgreSQL returns the result of the database operation.
6. The backend returns a response to the frontend.
7. The interface displays the result to the user.

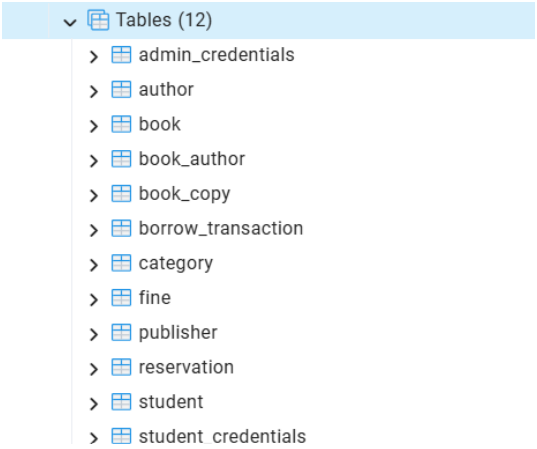


7.3 Database Creation and Configuration

The database is created in PostgreSQL and configured for use by the backend application.

The backend connection configuration specifies the database URL and the credentials required to access the database. These values must match the local PostgreSQL installation.

The database schema is responsible for defining the relations, data types, primary keys, foreign keys, and other constraints. Sample records can then be inserted to support development and testing.



7.4 Backend Implementation

The backend is implemented using Java and Spring Boot. It provides the application endpoints required by the frontend and handles operations involving library records.

For example, the student-management functionality includes retrieving student records, viewing student details, and deleting student records subject to backend checks.

The student deletion operation checks whether the student has related borrowing history or reservations before allowing deletion. Where related records prevent deletion, the backend can return a conflict response instead of deleting the student.

This approach helps protect related data and prevents the interface from being the only layer responsible for enforcing important business rules.

7.5 Frontend Implementation

The frontend is built using HTML, CSS, and JavaScript. The interface is organized into views for the administrator and student roles.

JavaScript handles interface interactions, renders records, and invokes backend operations where integration has been implemented. CSS provides the visual layout and styling, while HTML defines the structural elements of the application.

The UI design document describes the use of card-based catalog displays, dashboard elements, and reporting screens.

7.6 CRUD Operations

CRUD stands for Create, Read, Update, and Delete.

Create

Create operations insert new records into the relevant database relations. For example, a student record can be created with the student's identifier, name, email, phone number, and department.

Read

Read operations retrieve records from the database. Examples include displaying student details, retrieving book records, and viewing borrowing information.

Update

Update operations modify existing records. Examples include updating student contact information or changing book details.

Delete

Delete operations remove records when permitted by the application's rules and database constraints. Student deletion is subject to checks for related borrowing history and reservations.

CRUD operations are fundamental to maintaining library information and are implemented through the application and its database access layer as supported by each module.

7.7 SQL Operations

The following examples illustrate important SQL operations associated with the relational design. They should be executed only against a database whose table and column names match the final schema.

Example 1: Retrieve All Students

```
SELECT Student_ID, Name, Email, Phone, Department
FROM STUDENT;
```

Purpose: Retrieves student information for display in the student-management interface.

Example 2: Search for Books by Title

```
SELECT Book_ID, Title, ISBN, Publication_Year
FROM BOOK
WHERE Title ILIKE '%database%';
```

Purpose: Demonstrates case-insensitive title searching in PostgreSQL.

Example 3: Retrieve Books with Their Categories

```
SELECT B.Book_ID,
    B.Title,
    C.Category_Name
FROM BOOK B
JOIN CATEGORY C
  ON B.Category_ID = C.Category_ID;
```

Purpose: Demonstrates a join between BOOK and CATEGORY to retrieve related information.

Example 4: Retrieve Borrowing Information

```
SELECT BT.Borrow_ID,
    S.Name,
    BT.Issue_Date,
    BT.Due_Date,
    BT.Return_Date,
    BT.Status
FROM BORROW_TRANSACTION BT
JOIN STUDENT S
  ON BT.Student_ID = S.Student_ID;
```

Purpose: Retrieves borrowing transactions together with the corresponding student information.

Example 5: Count Books in Each Category

```
SELECT C.Category_Name,
    COUNT(B.Book_ID) AS Total_Books
FROM CATEGORY C
LEFT JOIN BOOK B
  ON C.Category_ID = B.Category_ID
GROUP BY C.Category_ID, C.Category_Name;
```

Purpose: Demonstrates the use of a join and an aggregate function to count book titles by category.

Important: These examples follow the submitted relational design. Verify the actual table names, columns, and constraints in the final PostgreSQL database before including the queries as executed implementation evidence. Add screenshots of the actual SQL output.

7.8 Version Control and Repository

Git and GitHub are used to maintain the project source code and supporting files. The repository contains the application files and database-related materials needed for project development and review.

The repository is available at:

https://github.com/adithyanbalu/BookVerse-SmartLibrary

Version control supports collaboration among team members and makes it easier to track changes to the application.

8. TESTING AND VALIDATION

8.1 Testing Objective

Testing is performed to determine whether the application behaves as expected, whether the frontend and backend communicate correctly, and whether the database maintains valid relationships between records.

The testing process should cover normal operations, invalid inputs, database constraints, and exceptional situations.

The following test cases provide a validation plan for the final application. Actual results must be recorded after running the corresponding tests.

8.2 Functional Test Cases

Test ID	Test Case	Expected Result	Actual Result
TC01	Open the application login page	Login interface is displayed	PASS
TC02	Enter valid administrator credentials	Authorized administrator access is provided	PASS
TC03	Retrieve the student list	Existing student records are displayed	PASS
TC04	Open a student's details	Correct details for the selected student are displayed	PASS
TC05	Delete an eligible student	Student is deleted and the displayed list is updated	PASS
TC06	Attempt to delete a student with related borrowing history or reservations	Deletion is prevented according to the backend rule	PASS
TC07	Retrieve the book catalog	Book records are displayed	PASS

TC08	Search for a book by title	Matching records are returned	PASS
TC09	Create or update a record through a supported form	Valid changes are saved and reflected in the interface	PASS
TC10	Open the reporting interface	The implemented report view is displayed	PASS

8.3 Database Constraint Validation

The following checks should be performed against the final schema.

Primary-Key Validation

Attempting to insert a duplicate primary-key value should fail.

Unique-Constraint Validation

Attempting to insert a duplicate value into a column protected by a unique constraint, such as a student email or book ISBN, should fail.

Foreign-Key Validation

Attempting to insert a child record that references a nonexistent parent record should fail when the relevant foreign-key constraint is enforced.

Required-Field Validation

Attempting to insert NULL into a required column should fail when the column is declared NOT NULL.

Referential Integrity Validation

Attempting to delete a record referenced by dependent records should behave according to the defined foreign-key rules and application-level checks.

8.4 Frontend and Backend Integration Testing

Integration testing verifies that the user interface communicates correctly with the Spring Boot backend and that the backend communicates correctly with PostgreSQL.

For example, when the student list is opened, the frontend should request the relevant records and display the returned data. When a deletion operation is performed, the backend should process the request, apply the required checks, and return a suitable response.

The frontend should display a meaningful success or error message based on the backend response.

8.5 Testing Summary

The final testing summary should be completed after all test cases have been executed.

Testing Category	Purpose
Functional testing	Verifies supported application operations
Database testing	Verifies records, constraints, and relationships
Integration testing	Verifies communication between frontend, backend, and database
Validation testing	Verifies handling of invalid inputs and exceptional situations
Regression testing	Checks that existing features continue to work after changes

The functional testing phase covered ten test cases involving application access, administrator authentication, student record retrieval, student details, student deletion, book catalog retrieval, book search, record creation or updating, and access to the reporting interface. All ten test cases were recorded as PASS based on the observed results.

The testing focused on verifying the expected behavior of the implemented application features. Additional database constraint and integration checks should be reported separately according to the tests actually performed and their observed outcomes.

9. RESULTS AND SCREENSHOTS

9.1 Results

The BookVerse Smart Library Management System was evaluated through functional testing of its implemented features. The testing covered administrator login, student record retrieval and details, student deletion validation, book catalog access, book search, record creation or updating, and access to the reporting interface. The database and backend were also examined using SQL query results and application logs.

The screenshots in this section provide evidence of database table verification, foreign-key relationship inspection, student credential status retrieval, and backend startup with database connectivity. Together, these results demonstrate the integration of the web interface, backend application, and relational database for the tested operations.

9.2 Database and Backend Verification

```
SELECT
    s.student_id,
    s.name,
    s.email,
    CASE
        WHEN sc.student_id IS NOT NULL THEN 'Credentials exist'
        ELSE 'Password not set'
    END AS login_status
FROM student s
LEFT JOIN student_credentials sc
    ON s.student_id = sc.student_id
ORDER BY s.student_id;
```

Data Output | Messages | Notifications

Showing rows: 1 t

	student_id [PK] integer	name character varying (100)	email character varying (150)	login_status text
1	25014	Adithyan B	adithyan@example.com	Credentials exi...
2	25024	Ali Adnan Thaha	ali@example.com	Password not ...
3	25040	Madhav A	madhav@example.com	Password not ...
4	25049	Robin Antony	robin@example.com	Password not ...
5	25051	Hrishi	hrishi@example.com	Credentials exi...

This shows the execution of an SQL query that retrieves student IDs, names, and email addresses along with their login credential status. A LEFT JOIN is used to associate student records with the corresponding entries in the student_credentials table. The CASE expression identifies whether credentials exist or whether a password has not yet been set. This query helps verify the relationship between student records and login credential records.

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_type = 'BASE TABLE'
  AND (
      table_name ILIKE '%user%'
      OR table_name ILIKE '%auth%'
      OR table_name ILIKE '%login%'
      OR table_name ILIKE '%credential%'
  )
ORDER BY table_name;
```

Data Output | Messages | Notifications

	table_name name
1	admin_credentials
2	author
3	book_author
4	student_credenti...

This screenshot shows an SQL query against PostgreSQL's information_schema.tables view. The query retrieves the names of selected base tables in the public schema whose names relate to users, authors, login information, or credentials. The results confirm the presence of tables such as admin_credentials, author, book_author, and student credential storage.

```sql
SELECT
    tc.table_name,
    kcu.column_name,
    ccu.table_name AS referenced_table,
    ccu.column_name AS referenced_column,
    tc.constraint_name
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
    ON tc.constraint_name = kcu.constraint_name
   AND tc.table_schema = kcu.table_schema
JOIN information_schema.constraint_column_usage AS ccu
    ON ccu.constraint_name = tc.constraint_name
   AND ccu.table_schema = tc.table_schema
WHERE tc.constraint_type = 'FOREIGN KEY'
  AND tc.table_schema = 'public'
  AND tc.table_name IN (
      'book',
```

Data Output | Messages | Notifications

Showing rows: 1 to 10

	table_name name	column_name name	referenced_table name	referenced_column name	constraint_name name
1	book	category_id	category	category_id	fk_book_category
2	book	publisher_id	publisher	publisher_id	fk_book_publisher
3	book_author	author_id	author	author_id	book_author_author_id_fkey
4	book_author	book_id	book	book_id	book_author_book_id_fkey
5	book_copy	book_id	book	book_id	book_copy_book_id_fkey
6	borrow_transacti...	accession_numb...	book_copy	accession_number	borrow_transaction_accession_number_f...
7	borrow_transacti...	student_id	student	student_id	borrow_transaction_student_id_fkey
8	fine	borrow_id	borrow_transacti...	borrow_id	fine_borrow_id_fkey
9	reservation	book_id	book	book_id	reservation_book_id_fkey
10	reservation	student_id	student	student_id	reservation_student_id_fkey

This displays the foreign key relationships defined in the BookVerse database. The query retrieves the referencing table, referencing column, referenced table, referenced column, and constraint name from PostgreSQL's information schema. The results show relationships involving books, categories, publishers, authors, book copies, borrowing transactions, students, fines, and reservations. These constraints help maintain referential integrity between related records.

```
at.TomcatWebServer  : Tomcat started on port 8080 (http) with context path '/'
2026-10-11T13:02:53.605+05:30  INFO 26668 --- [bookverse-backend] [           main] c.
ackendApplication   : Started BackendApplication in 2.801 seconds (process running for
==============================
BOOKVERSE DATABASE CONNECTED!
Total books: 8
==============================
2026-10-11T13:07:19.651+05:30  INFO 26668 --- [bookverse-backend] [nio-8080-exec-1] o.
ocalhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-11T13:07:19.652+05:30  INFO 26668 --- [bookverse-backend] [nio-8080-exec-1] o.
```

This shows the Spring Boot backend starting successfully and reporting that the BookVerse database is connected. The application also reports a total of eight books. The log indicates that the embedded Tomcat server started on port 8080 and that an incoming HTTP request triggered the Spring DispatcherServlet. This provides evidence that the backend application started and established database connectivity during the displayed run.

10. CHALLENGES FACED AND SOLUTIONS

10.1 Database Relationship Design

Challenge: The library contains several related entities, including books, authors, students, physical copies, borrowing transactions, reservations, and fines. Representing these relationships without unnecessary duplication requires careful database design.

Solution: The project uses separate relations for the principal entities and foreign keys to connect related records. The BOOK_AUTHOR junction table represents the many-to-many relationship between books and authors.

10.2 Normalization and Redundancy

Challenge: Storing repeated information in multiple tables can lead to update inconsistencies and data anomalies.

Solution: The relational design separates entities such as CATEGORY, PUBLISHER, and AUTHOR from BOOK. The corresponding identifiers are referenced through foreign keys, reducing repeated descriptive information.

10.3 Frontend and Backend Integration

Challenge: The frontend must communicate with the backend to retrieve and modify persistent data. Errors in request handling, response processing, or endpoint accessibility can prevent the interface from displaying the expected results.

Solution: The application separates frontend presentation from backend request processing. Integration testing is used to verify that supported operations return the expected data or error responses.

10.4 Student Deletion and Referential Integrity

Challenge: Deleting a student who has related borrowing history or reservations can leave inconsistent records or violate database constraints.

Solution: The student-management backend checks for related borrowing history and reservations before allowing deletion. This prevents deletion when the defined business rules do not permit it.

10.5 Database Configuration

Challenge: The backend requires a valid connection to PostgreSQL. Incorrect database URLs, credentials, or schema definitions can prevent the application from starting or retrieving data.

Solution: The database connection configuration must be checked against the local PostgreSQL installation. The database schema and sample data must also be consistent with the application code.

10.6 Testing and Error Handling

Challenge: A feature that appears correct in the user interface may still fail because of backend validation, database constraints, or unexpected responses.

Solution: Testing should cover both successful and unsuccessful operations. Backend responses and database results should be checked alongside the frontend behavior.

11. CONCLUSION AND FUTURE ENHANCEMENTS

11.1 Conclusion

The Smart Digital Library Management System, BookVerse, is a database-driven project developed to demonstrate the practical use of relational database concepts in a college library environment.

The project identifies the principal entities involved in library operations and organizes them into related relational tables. The database design uses primary keys, foreign keys, functional dependencies, and normalization principles to improve data organization and reduce redundancy.

The application uses HTML, CSS, and JavaScript for the frontend, Java with Spring Boot for backend processing, and PostgreSQL for persistent data storage. This layered approach separates presentation, application logic, and database responsibilities.

The student-management workflow demonstrates how application-level validation can protect related records during deletion. The project also provides a foundation for catalog management, borrowing records, reservations, and fine management according to the designed data model.

Overall, BookVerse demonstrates how DBMS principles can be applied to a practical application and provides opportunities for further improvements in automation, reporting, and data management.

11.2 Future Enhancements

The following enhancements can be considered for future versions:

1. **Automated Fine Calculation:** Implement and validate fine calculation based on due dates, return dates, and configurable fine rules.
2. **Advanced Reporting:** Provide detailed reports on borrowing activity, popular categories, overdue transactions, and collection status.
3. **Improved Authentication:** Strengthen authentication and authorization mechanisms for administrator and student accounts.
4. **Notifications:** Add notifications for due dates, overdue books, and reservation status changes.

5. **Enhanced Search:** Improve catalog search with additional filters and more efficient database queries.
6. **Transaction Management:** Ensure that related operations execute consistently using appropriate database transactions.
7. **Backup and Recovery:** Introduce regular database backups and documented recovery procedures.
8. **Deployment:** Deploy the backend and database using a suitable hosting environment with secure configuration.
9. **Audit Logging:** Maintain records of important administrative operations to improve accountability.
10. **Mobile-Friendly Access:** Further improve the interface for users accessing the system through mobile devices.

These enhancements can extend the system's functionality while preserving the relational database design and the separation between the frontend, backend, and database.

REFERENCES

1. PostgreSQL Global Development Group. *PostgreSQL Documentation*. https://www.postgresql.org/docs/
2. Spring. *Spring Boot Reference Documentation*. https://docs.spring.io/spring-boot/reference/
3. Oracle. *Java Documentation*. https://docs.oracle.com/en/java/
4. MDN Web Docs. *HTML: HyperText Markup Language*. https://developer.mozilla.org/en-US/docs/Web/HTML
5. MDN Web Docs. *CSS: Cascading Style Sheets*. https://developer.mozilla.org/en-US/docs/Web/CSS
6. MDN Web Docs. *JavaScript*. https://developer.mozilla.org/en-US/docs/Web/JavaScript
7. GitHub. *BookVerse – Smart Digital Library Management System*. https://github.com/adithyanbalu/BookVerse-SmartLibrary
8. *Smart Digital Library Management System – Project Abstract, Group A8.*
9. *Smart Digital Library Management System – UI Design Document, Group A8.*
10. *Smart Digital Library Management System – Database Design Document, Group A8.*